

TKOM Dokumentacja finalna - PyScript

Andrzej Pultyn

Semestr 25L

Spis treści

1	Zadanie	2
2	Wymagania projektu	2
2.1	Wymagania funkcjonalne	2
2.2	Wymagania niefunkcjonalne	3
3	Opis realizowanych funkcjonalności	3
3.1	Charakterystyka języka	3
3.2	Typy danych	3
3.2.1	Rodzaje typów danych	3
3.2.2	Lista typów danych	3
3.3	Zmienne	6
3.4	Wyrażenia arytmetyczne i logiczne	6
3.5	Instrukcje warunkowe	7
3.6	Pętle	8
3.7	Własne funkcje	8
3.8	Funkcje wbudowane	9
3.9	Obiektość	9
3.10	Komentarze	10
3.11	Słownik	10
3.11.1	Charakterystyka	10
3.11.2	Tworzenie nowego słownika	10
3.11.3	Metody	11
3.12	Zapytania LINQ	11
4	Obsługa błędów	12
5	Sposób uruchomienia	14
6	Składnia realizowanego języka	14
6.1	Słowa kluczowe języka	14
6.2	Gramatyka	15
7	Testowanie	17

1 Zadanie

Zadaniem było napisanie interpretera do języka z wbudowanym typem słownika z określoną kolejnością elementów. Kolejność elementów w słowniku jest określana w momencie jego tworzenia - argumentem konstruktora słownika jest funkcja określająca, w jakiej kolejności mają być dodawane elementy. Możliwe powinny być podstawowe operacje na słowniku (dodawanie, usuwanie, wyszukiwanie elementów według klucza, sprawdzenie, czy dany klucz znajduje się w słowniku itd.), iterowanie po elementach oraz wykonywanie na słowniku zapytań w stylu LINQ.

2 Wymagania projektu

2.1 Wymagania funkcjonalne

Język powinien zawierać podstawowe elementy języka programowania oraz umożliwiać wykonanie podstawowych operacji programistycznych:

- typy całkowitoliczbowe oraz zmiennoprzecinkowe
- stałe znakowe
- wyrażenia arytmetyczne i logiczne
- instrukcje warunkowe
- pętle
- funkcje (wraz z rekursywnym wywołaniem)
- namiastka obiektowości (dodatkowe typy danych z konstruktorami, polami/metodami, dostępem do składowych przez kropkę)
- komentarze
- zmienne

Oprócz podstawowych elementów, język powinien posiadać klasę słownika o następujących funkcjonalnościach:

- przechowywanie par klucz-wartość
- dodawanie elementów
- usuwanie elementów
- wyszukiwanie elementów według klucza
- sprawdzanie, czy dany klucz znajduje się w słowniku
- iterowanie po elementach słownika
- wykonywanie zapytań w stylu LINQ

Interpreter powinien zapewniać:

- obsługę błędów języka z wartościowymi informacjami zwrotnymi
- czytanie kodu z plików

2.2 Wymagania niefunkcjonalne

- wygoda pisania i czytania kodu
- brak kończenia działania interpretera na skutek niespodziewanego błędu

3 Opis realizowanych funkcjonalności

3.1 Charakterystyka języka

Język jest dynamicznie oraz silnie typowany. Bloki kodu opierają się na nawiasach klamrowych, tak jak w języku C czy Java. Białe znaki, nie licząc stałych tekstowych i komentarzy, nie mają znaczenia. Każde wyrażenie kończy się średnikiem.

3.2 Typy danych

3.2.1 Rodzaje typów danych

Język zawiera 2 rodzaje danych - proste i złożone. Do prostych należą typy *Int*, *Float*, *String* oraz *Bool*. Przekazywane są jako argumenty wywołania funkcji **przez wartość**. Zmienne przechowują tylko wartość, która jest automatycznie kopiowana:

```
{
    a = 5;
    b = a; // przypisana jest KOPIA wartości pod a
    b += 5; // wartość a pozostaje niezmienną
}
```

Do typów złożonych należą *Item*, *List*, *Dict* oraz typ funkcji. Te typy danych są przechowywane w zmiennych i przekazywane do funkcji jako referencja. Zawierają także metodę **copy()**, która zwraca płytką kopię obiektu - tzn. jeżeli typ złożony zawiera referencję do innego złożonego obiektu, to skopiowany obiekt będzie zawierać tę samą referencję:

```
{
    a = [1, 2, 3];
    b = a; // przypisana jest REFERENCJA do tej samej listy

    b.add(4);

    // a = [1, 2, 3, 4]
    // b = [1, 2, 3, 4]
}
```

3.2.2 Lista typów danych

- **Int** - typ liczb stałoprzecinkowych:

```
{
    a = 1;
    b = -15 + 38;
}
```

Każda wartość *Int* ma 2 metody wbudowane:

- **toFloat()** - rzutująca wartość na typ *Float*
 - **toString()** - rzutująca wartość na typ *String*
- **Float** - typ liczb zmiennoprzecinkowych. Wartości zmiennoprzecinkowe **muszą** zawierać kropkę, oraz przynajmniej jedną cyfrę po przecinku:

```
{
    a = 1.0;
    b = -15.38;
}
```

Analogicznie do typu *Int*, typ *Float* posiada metody wbudowane:

- **toInt()** - rzutująca wartość na typ *Int*, obcinająca wartość po przecinku
- **toString()** - rzutująca wartość na typ *String*

Użycie funkcji wbudowanej na literale typu *Float* wymaga wzięcia literału w cudzysłów:

```
{
    9.5.toInt();
    // błąd, traktowane jako literal float z wieloma kropkami
    (9.5).toInt(); // prawidłowo, zwrócone zostanie 9
}
```

- **String** - typ stałych znakowych. Stałe umieszczone są w cudzysłowie:

```
{
    a = "Hello World!";
    a = "Typing with quotes: \"\" and backslash: \\";
}
```

Język obsługuje escaping znaków, dodając znak backslash. Typ *String* posiada następujące metody:

- **length()** - zwracająca długość napisu
- **toFloat()** - próbująca rzutować tekst na wartość *Float*
- **toInt()** - próbująca rzutować tekst na wartość *Int*

- **Bool** - *boolean* - typ boolowski:

```
{
    a = True;
    b = False;
    c = 5 < 4;    // False
}
```

Typ *Bool* nie posiada wbudowanych metod

- **List** - typ nieuporządkowanej listy. Przyjmować może ona dowolne typy wartości z języka, również inne listy, tworząc listę zagnieżdżoną:

```
{
    empty = [];
    comples = [1, "Hello", -10.5, ("key": 5),
               ["hello", "from", "sublist"],
               {"name": "dict", "value": 5}]
}
```

Na typie listy można będzie wykonać następujące operacje:

- **get(indeks)** - pobranie elementu na podanym indeksie
 - **set(indeks, wartość)** - podmiana wartości na podanym indeksie na inną
 - **add(wartość)** - dodanie elementu na koniec listy
 - **remove(indeks)** - usunięcie elementu o podanym indeksie z listy
 - **toString()** - zwracająca tekstową reprezentację listy
 - **length()** - zwracający długość listy
 - **copy()** - zwracający płytką kopię listy
- **Dict** - *dictionary* - typ słownika. Zawierać może wyłącznie obiekty typu *Item*:

```
{
    {};
    {
        ("first_key": 5),
        ("another_key": "value"),
        ("one_more": [1, 5, "hello"]),
        ("last_one": {"key": "value"})
    };
}
```

Funkcjonalności słownika opisane w dedykowanym podrozdziale.

- **Item** - typ obiektu przechowywanego w słowniku. Zawiera parę klucz oraz wartość. Klucz musi być typem prostym, a wartość może być dowolna:

```
{
    ("key": "value");
    (1: 10);
}
```

Typ zawiera metody:

- **key()** - zwracająca klucz
 - **value()** - zwracająca wartość
 - **toString()** - zwracająca reprezentację tekstową obiektu
 - **copy()** - zwracająca płytką kopię obiektu
- **Funkcja** - funkcja również jest typem danych, który może być przypisywany do zmiennych, przekazywany do innych funkcji, oraz zwracany jako wartość.

3.3 Zmienne

Zmienne definiowane są podaniem identyfikatora, a następnie przypisaniem do niej wartości:

```
{
    a = "Hello there";
    my_list = [1, "Hello", a];
}
```

Zakres widoczności zmiennych opiera się na strukturze Env, która posiada listę swoich symboli oraz wskazanie na środowisko poprzedzające. W poszukiwaniu wartości identyfikatora, przeszukiwane jest najpierw obecne środowisko, a później kolejno odwiedzani są rodzice. Zmienne przechowują wartość lub wskazanie na obiekt, w zależności od typu danych (patrz punkt 3.2.1).

3.4 Wyrażenia arytmetyczne i logiczne

Wyrażenia mogą być stałymi odpowiednich typów, wartościami zwracanymi przez funkcje, lub ich kombinacjami z operatorami:

Operator	Znaczenie	Priorytet	Asocjacja
()	Zawołanie obiektu	9	Lewostronna
.	Składowa obiektu	8	Lewostronna
!	Negacja logiczna	7	Prawostronna
-	Negacja arytmetyczna	7	Prawostronna
*	Mnożenie arytmetyczne	6	Lewostronna
/	Dzielenie arytmetyczne	6	Lewostronna
+	Dodawanie arytmetyczne	5	Lewostronna
-	Odejmowanie arytmetyczne	5	Lewostronna
>	Większy niż	4	Lewostronna
>=	Większy lub równy	4	Lewostronna
<	Mniejszy niż	4	Lewostronna
<=	Mniejszy lub równy	4	Lewostronna
==	Równy	4	Lewostronna
!=	Nierówny	4	Lewostronna
and	Logiczny AND	3	Lewostronna
or	Logiczny OR	2	Lewostronna
=	Operator przypisania	1	Prawostronna
+=	Operator przypisania	1	Prawostronna
-=	Operator przypisania	1	Prawostronna

Kilka ważnych uwag dotyczących operatorów:

- operatory działań arytmetycznych (+, -, *, /) wymagają identycznych typów liczbowych po obu stronach (*Int* lub *Float*)
- operator dodawania działa również na typach *String*, *List* jako konkatencja
- operatory porównania (==, !=, <, <=, >, >=) działają również na typie string. Równość i nierówność sprawdzają, czy stringi są identyczne, a większy/mniejszy rozstrzygany jest poprzez porządek alfabetyczny
- operatory równości i nierówności (==, !=) działają na różnych typach danych, gdzie różne typy danych traktowane są jako nierówne obiekty

3.5 Instrukcje warunkowe

Przy tworzeniu instrukcji warunkowych, można skorzystać z 3 poleceń, znanych z innych języków programowania: *if*, *elif*, *else*:

```
{
    if (a < 4) {
        do_something();
    } elif (a > 4) {
        do_something_else();
    } else {
        do_something_differently();
    }
}
```

3.6 Pętle

Pętle skonstruować będzie można na 2 sposoby:

- słowem *while*, która będzie wykonywana dopóki podany warunek jest prawdziwy:

```
{
    a = 0;
    while (a < 10) {
        do_something_ten_times();
        a += 1;
    }
}
```

- słowem *for*, która służy do iteracji po elementach listy lub słownika:

```
{
    my_dict = {"first": 1}, {"second": 10};
    for element in my_dict {
        print(element.key()); // "first" "second"
    }
}
```

Iterujemy po elementach po kolei. W przypadku słownika, elementy są sortowane podczas dodawania elementów, więc iteracja po kolei da odpowiednią kolejność.

3.7 Własne funkcje

Funkcje definiujemy, przypisując ją do zmiennej:

```
{
    my_func = function(arg1, arg2) {
        return arg1 + arg2;
    }

    print(my_func(5, 10)) // 15;
}
```

Funkcje mogą być wywoływane rekursywnie:

```
{
    factorial = function(n) {
        if (n == 1) {
            return 1;
        }
        return n * factorial(n-1);
    }
}
```

Oraz mogą być przekazywane do innych funkcji jako argumenty:


```

{
    passed_func = function(a, b) {
        return a + b;
    }

    another_function = function(func, a) {
        return func(a, 5);
    }

    print(another_function(passed_func, 10));
    // 15
}

```

Sposób przekazywania wartości do funkcji opisany w punkcie 3.2.1.

3.8 Funkcje wbudowane

Język będzie zawierać następujące funkcje wbudowane:

- metody wbudowane dla obiektów, opisane w punkcie 3.2.2
- **typeof(variable)** - funkcja zwracająca *stringa* z nazwą typu obiektu
- **Dict(func)** - konstruktor słownika, któremu należy podać funkcję sortującą
- **print(wartość1, wartość2, ...)** - wypisanie na standardowe wyjście podanych wartości. Elementy złożone takie jak *List* czy *Dict* również mogą być wypisywane. W przypadku zagnieżdżonych elementów złożonych, wypisywany będzie ich typ wraz z jego długością:

```

{
    my_list = [
        1, 5, [1, "Hello", {"key", 5}],
        {"key", 2}, ("diff", {})
    ];

    print(my_list);
    // [1, 5, List(3), Dict(1)]

    print(my_list.get(3);
    // {"key", 2}, ("diff", Dict(0))

    print(my_list.get(2);
    // [1, "Hello", Dict(1)]
}

```

3.9 Obiektość

Język zawiera elementy obiektości, mamy specjalne typy (*Item*, *Dict*, *List*), które zawierają funkcje składowe (np. *List.get(idx)*). Mamy również konstruktor do klasy *Dict*.

3.10 Komentarze

Komentarze w języku można pisać na 2 sposoby, identyczne jak w języku C:

- podwójny slash - tworzący komentarz do końca linii
- `/* */` - komentarz inline od jednego znaku do drugiego

Będą one ignorowane jako *whitespace*.

3.11 Słownik

3.11.1 Charakterystyka

Słownik jest kolekcją, przechowującą elementy typu *Item* o unikalnych kluczach. Przy tworzeniu słownika można podać mu funkcję sortującą, decydującą o kolejności elementów. Powinna ona być standardową funkcją porównawczą, czyli przyjmować 2 elementy typu *Item*, oraz zwracać:

- -1, jeżeli pierwszy element powinien być wcześniej niż drugi
- 0, jeżeli elementy są równe
- 1, jeżeli pierwszy element powinien być później niż drugi

Przy dodawaniu nowego elementu do słownika, zostanie porównany po kolei z każdym elementem i dodany przed pierwszym elementem, z którym porównanie da wynik -1, lub na koniec słownika.

3.11.2 Tworzenie nowego słownika

Słownik można stworzyć na 2 sposoby:

- poprzez literał - tworzymy słownik poprzez nawias klamrowy i wpisując jego zawartość. Słownik wtedy przyjmuje domyślną funkcję sortującą (zwracającą zawsze 1). Wtedy elementy są posortowane według kolejności dodawania do słownika:

```
{
    literal_dict = {"first": "Hello", ("second": 10)};

    literal_dict.add("another", 5);
    // element dodany na koniec słownika
}
```

- poprzez konstruktor - tworzymy nowy słownik,wołając jego konstruktor. Jako argument podajemy własną funkcję sortującą:

```
{
    /* funkcja sortująca w odwrotnej kolejności
       alfabetycznej kluczy */
    my_sort = function(item1, item2) {
        if (item1.key() < item2.key()) {
            return 1;
        }
    }
```

```

        if (item1.key() > item1.key()) {
            return -1;
        }
        return 0;
    }

    my_dict = Dict(my_sort);

    my_dict.add("key", 5);
    my_dict.add("zebra", "hello");
    // element dodany na początek słownika, zgodnie z regułą
}

```

3.11.3 Metody

Słownik zawierać będzie następujące funkcje wbudowane:

- **addNew(key, value)** - utworzenie nowego obiektu *Item* i dodanie go do słownika (zgłaszany błąd, jeżeli słownik już posiada obiekt o podanym kluczu)
- **add(item)** - dodanie istniejącego obiektu *Item* do słownika (również zgłaszany błąd, jeżeli słownik już posiada obiekt o kluczu równym kluczowi podanego obiektu)
- **remove(key)** - usunięcie ze słownika elementu o podanym kluczu (zgłaszany błąd, jeżeli słownik nie posiada takiego obiektu)
- **contains(key)** - sprawdzenie, czy słownik zawiera element o podanym kluczu (zwraca wartość boolowską)
- **get(key)** - uzyskanie obiektu o podanym kluczu (zgłaszany błąd, jeżeli słownik nie posiada obiektu o podanym kluczu)
- **toString()** - zwraca tekstową reprezentację słownika
- **length()** - zwraca długość słownika

3.12 Zapytania LINQ

Na słowniku i liście można wykonywać również zapytania LINQ o konstrukcji:

```

from <var> in <source>
    select <selects>, <selects>, ... <selects>
    where <condition>
    order by <expression>
    <ewentualnie descending>;

```

- **var** - identyfikator, pod którym dostępne są po kolei elementy z kolekcji (jeżeli wcześniej mieliśmy zdefiniowany taki identyfikator, to zostanie przykryty)
- **source** - wyrażenie zwracające źródło danych (listę lub słownik)
- **selects** - wyrażenia, które mają zostać zwrócone dla każdego elementu ze źródła

- **condition** - warunek decydujący, czy dany element z kolekcji zostanie wzięty pod uwagę przy obliczaniu *selects*
- **order by expression** - wyrażenie zwracające wartość dla każdego elementu z kolekcji, po której wyniki będą sortowane
- **descending** - odwrócenie kolejności wyników (wymagana klauzula *order by*)

Zapytania zwracają **listę list** wyrażen zwracanych dla każdego wybranego elementu:

```
{
  my_dict = {
    ("Warszawa", 2000000),
    ("Tokio", 37000000),
    ("Delhi", 30000000),
    ("Szczecinek", 40000),
    ("Buenos Aires", 15000000)
  }

  small_cities = from city in my_dict
    select city.key(), city.value() / 1000
    where city.value() < 5000000
    order by city.key() descending;

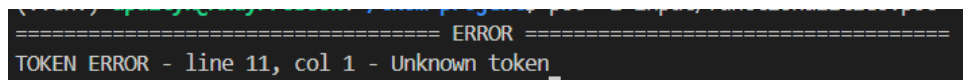
  // small_cities = [{"Warszawa", 2000}, {"Szczecinek", 40}]
}
```

4 Obsługa błędów

Na każdym etapie interpretowania kodu może występować błąd, przekazywany do modułu obsługi błędów. Błędy w ogólności wyglądają następująco:

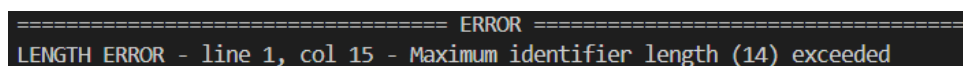
<Rodzaj błędu> ERROR - line <wiersz>, col <kolumna> - <wiadomość>

Przykładowe błędy leksera:



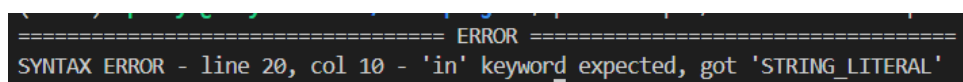
```
===== ERROR =====
TOKEN ERROR - line 11, col 1 - Unknown token
```

Rysunek 1: Błąd leksera - nieznany token



```
===== ERROR =====
LENGTH ERROR - line 1, col 15 - Maximum identifier length (14) exceeded
```

Rysunek 2: Błąd leksera - zbyt długi identyfikator



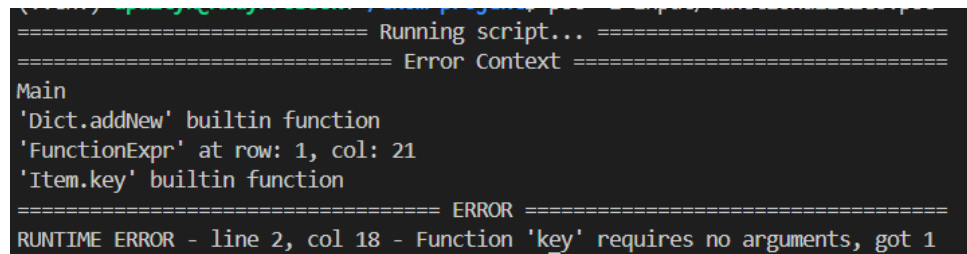
```
===== ERROR =====
SYNTAX ERROR - line 20, col 10 - 'in' keyword expected, got 'STRING_LITERAL'
```

Rysunek 3: Błąd parsera - otrzymano inny token niż spodziewany

Błędy w trakcie wykonywania kodu (*Runtime Exception*) dodatkowo mają wypisywany kontekst wystąpienia błędu. Przykładowo, w poniższym kodzie, funkcja sortująca *alphabetical_sort* zawiera wywołanie funkcji *key*, która nie przyjmuje żadnych argumentów, a jeden został podany:

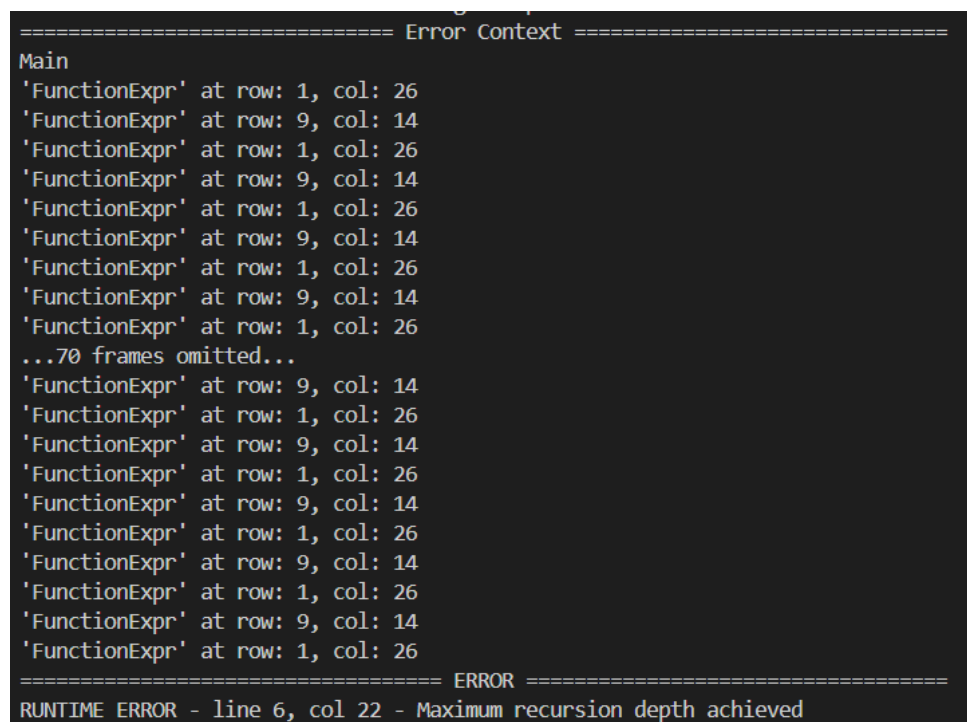
```
alphabetical_sort = function(item1, item2) {
    if (item1.key(1) < item2.key()) {
        return -1;
    }
    if (item1.key() > item2.key()) {
        return 1;
    }
    return 0;
};
```

```
cities = Dict(alphabetical_sort);
cities.addNew("Warszawa", 2000000);
```



```
===== Running script... =====
===== Error Context =====
Main
'Dict.addNew' builtin function
'FunctionExpr' at row: 1, col: 21
'Item.key' builtin function
===== ERROR =====
RUNTIME ERROR - line 2, col 18 - Function 'key' requires no arguments, got 1
```

Rysunek 4: Błąd wraz z kontekstem wystąpienia



```
===== Error Context =====
Main
'FunctionExpr' at row: 1, col: 26
'FunctionExpr' at row: 9, col: 14
'FunctionExpr' at row: 1, col: 26
'FunctionExpr' at row: 9, col: 14
'FunctionExpr' at row: 1, col: 26
'FunctionExpr' at row: 9, col: 14
'FunctionExpr' at row: 1, col: 26
'FunctionExpr' at row: 9, col: 14
'FunctionExpr' at row: 1, col: 26
'FunctionExpr' at row: 9, col: 14
...70 frames omitted...
'FunctionExpr' at row: 9, col: 14
'FunctionExpr' at row: 1, col: 26
'FunctionExpr' at row: 9, col: 14
'FunctionExpr' at row: 1, col: 26
'FunctionExpr' at row: 9, col: 14
'FunctionExpr' at row: 1, col: 26
'FunctionExpr' at row: 9, col: 14
'FunctionExpr' at row: 1, col: 26
'FunctionExpr' at row: 9, col: 14
'FunctionExpr' at row: 1, col: 26
===== ERROR =====
RUNTIME ERROR - line 6, col 22 - Maximum recursion depth achieved
```

Rysunek 5: Błąd rekurencji

5 Sposób uruchomienia

Programy można uruchamiać na 2 sposoby:

- z pliku .psc:

```
$ psc -i program.psc
> Hello from file!
```

Plik program.psc:

```
print("Hello World");
```

- ze strumienia tekstowego:

```
$ echo "a = 10; print(a + 10);" | psc
> 20
```

Interpreter będzie miał specjalną flagę `-c`, po której należy podać plik konfiguracyjny `.json`. Umożliwia to modyfikowanie parametrów poszczególnych narzędzi. Na razie możliwe są poniższe modyfikacje (szablon do pliku konfiguracyjnego):

```
{
  "lexer": {
    "max_comment_length": 10000
    "max_identifier_length": 1000
    "max_string_literal_length": 10000
    "max_num_literal_length": 100
  }
}
```

```
$ psc program.psc -c config.json
> Hello from file with configured interpreter!
```

6 Składnia realizowanego języka

6.1 Słowa kluczowe języka

- and
- or
- if
- elif
- else
- True

- False
- function
- return
- while
- for
- from
- in
- select
- where
- order
- by
- descending

6.2 Gramatyka

```

program    = { statement }, "EOF" ;

statement  = stmt_with_ident,
            | if_statement
            | while_loop
            | for_loop
            | return_statement
            | block ;

stmt_with_ident = identifier, ( assignment | call_stmt ), ";" ;
assignment     = ( "=" | "+=" | "-=" ), expression ;
call_stmt      = { access_suff | call_suff }, call_suff ;

access_suff    = ".", identifier ;
call_suff      = "(", [ expression, { ",", expression } ], ")"

if_statement   = "if", "(", expression, ")", block,
                { elif }, [ "else", block ] ;
elif           = "elif", "(", expression, ")", block ;

while_loop     = "while", "(", expression, ")", block ;
for_loop       = "for", identifier, "in", expression, block ;
return_statement = "return", [ expression ], ";" ;
block          = "{", { statement }, "}" ;

expression    = logic_or ;

```

```

logic_or  = logic_and, { "or", logic_and } ;
logic_and = comparison, { "and", comparison } ;
comparison = additive,
    { ("==" | "!=" | ">" | ">=" | "<" | "<="), additive } ;
additive  = multiplicative, { ("+" | "-"), multiplicative } ;
multiplicative = unary, { ( "*" | "/" ), unary } ;
unary      = { ("!" | "-") }, postfix ;
postfix    = primary, { call_suff | member_suff } ;

primary    = integer
    | float
    | string
    | "True" | "False"
    | identifier
    | list_literal
    | dict_literal
    | func_literal
    | linq_query
    | item_literal_or_parenthesis

list_literal = "[", [ expression, { ",", expression } ] "]" ;
dict_literal = "{", [ item_literal, { ",", item_literal } ], "}" ;
func_literal  = "function",
    "(", identifier, { ",", identifier }, ")", block ;

linq_query    = "from", identifier, "in", expression,
    "select", expression, { ",", expression }
    [ "where", expression ],
    [ "order", "by", expression, [ "descending" ] ] ;

item_literal_or_parenthesis = "(", expression, ( item_literal_tail | ")" ) ;
item_literal                = "(", expression, item_literal_tail ;
item_literal_tail           = ":", expression, ")" ;

integer    = "0" | ( digit-positive , { digit } ) ;
float      = integer, ".", integer ;
string     = "'", { unicode_symbol }, "'" ;
identifier = letter, { alphanumeric } ;

unicode_symbol = { *symbole UNICODE* } ;
letter         = { *litory* } ;
alphanumeric   = letter | digit | "_" ;

comment-endline = "//", { unicode_symbol }, "\n";
comment-inline  = "/*", { unicode_symbol }, "*/";

digit-positive = "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9";
digit          = "0" | digit-positive;

```


Ostateczna wersja gramatyki zamieszczona jest również w repozytorium w pliku *final-grammar.txt*.

7 Testowanie

Każdy moduł interpretera (lekser, parser, interpreter) został zbadany testami jednostkowymi przy użyciu biblioteki *pytest*. Sprawdzone zostanie nie tylko poprawność działania, ale i poprawność zgłaszanych wiadomości i pozycji błędu. Przykładowy program wykorzystujący podstawowe funkcje języka:

```
alphabetical_sort = function(item1, item2) {
    if (item1.key(1) < item2.key()) {
        return -1;
    }
    if (item1.key() > item2.key()) {
        return 1;
    }
    return 0;
};

cities = Dict(alphabetical_sort);
cities.addNew("Warszawa", 2000000);
cities.addNew("Tokio", 37000000);
cities.addNew("Delhi", 30000000);
cities.addNew("Szczecinek", 40000);
cities.addNew("Buenos Aires", 15000000);

print("Miasta i populacja:");
for city "Hello" in cities {
    print("- ", city.key(), ":", city.value());
}

is_large = function(value) {
    return value > 5000000;
};

small_cities = from city in cities
    select city.key(), city.value().toFloat() / 1000000.0
    where !is_large(city.value())
    order by city.key() descending;

print("Małe miasta (< 5 mln):");
for entry in small_cities {
    print("- ", entry.get(0), " (", entry.get(1), " mln.)");
}

if (!cities.contains("Berlin")) {
    cities.addNew("Berlin", 3800000);
}
```

```

}

if (!cities.contains("Berlin")) {
    cities.addNew("Berlin", 3800000);
}

print("Typ cities:", typeof(cities));
print("Typ small_cities:", typeof(small_cities));

backup_list = small_cities.copy();
backup_list.add(["Kopenhagen", 0.638]);

print("Original: ", small_cities);
for entry in small_cities {
    print("- ", entry.get(0), " (", entry.get(1), " mln.)");
}

print("Backup: ", backup_list);
for entry in backup_list {
    print("- ", entry.get(0), " (", entry.get(1), " mln.)");
}

string_with_escaping = "Hello \"TKOM\" and backslash: \\";
print(string_with_escaping);

need_func = function(func, value) {
    return func(value);
};

func_to_pass = function(value) {
    return value + 5;
};

print(need_func(func_to_pass, 5));

print((9.5).toInt());

```